

Histogram sorting

```
#include <iostream>
#include <omp.h>

#define SIZE 7
#define MAX_VALUE 100

int main() {
    int input[SIZE] = {5, 2, 9, 5, 2, 0, 1};
    int histogram[MAX_VALUE + 1] = {0};
    int output[SIZE];

    // Step 1: Build histogram in parallel
    #pragma omp parallel
    {
        // Allocate a local histogram for each thread
        int* local_histogram = new int[MAX_VALUE + 1];

        // Initialize local histogram to 0
        for (int i = 0; i <= MAX_VALUE; ++i)
            local_histogram[i] = 0;

        // Each thread updates its own local histogram
        #pragma omp for
        for (int i = 0; i < SIZE; ++i)
            local_histogram[input[i]]++;

        // Merge local histograms into global histogram (critical section)
        #pragma omp critical
        {
            for (int i = 0; i <= MAX_VALUE; ++i)
                histogram[i] += local_histogram[i];
        }

        // Free dynamically allocated local histogram
        delete[] local_histogram;
    }

    // Step 2: Reconstruct the sorted output array
    int index = 0;
    for (int i = 0; i <= MAX_VALUE; ++i) {
        for (int j = 0; j < histogram[i]; ++j) {
            output[index++] = i;
        }
    }

    // Step 3: Print the sorted array
    std::cout << "Sorted array:\n";
    for (int i = 0; i < SIZE; ++i)
```

```
        std::cout << output[i] << " ";  
std::cout << "\n";  
return 0;  
}
```

BFS

```
#include <iostream>
#include <omp.h>
#include <queue>

#define V 7

using namespace std;

void BFS(int adj[V][V]) {
    bool visited[V] = {false}; // Track visited nodes
    queue<int> que;             // Queue for BFS traversal

    visited[0] = true;
    que.push(0);

    cout << "BFS Order:\n";

    #pragma omp parallel
    {
        while (true) {
            int node = -1;

            // Ensure only one thread pops from the queue at a time
            #pragma omp critical
            {
                if (!que.empty()) {
                    node = que.front();
                    que.pop();
                }
            }

            // If no nodes are left, break out of the loop
            if (node == -1) {
                break;
            }

            // Print the node (critical section ensures correct ordering of output)
            #pragma omp critical
            {
                cout << node << " ";
            }

            // Parallelize exploration of neighbors
            #pragma omp for
            for (int i = 0; i < V; i++) {
                if (adj[node][i] && !visited[i]) {
                    visited[i] = true; // Mark the node as visited
                    #pragma omp critical
                    {
```

```
int main() {
    // Example adjacency matrix for an undirected graph
    int adj[V][V] = {
        {0, 1, 1, 0, 0, 0, 0},
        {1, 0, 0, 1, 1, 0, 0},
        {1, 0, 0, 0, 0, 1, 1},
        {0, 1, 0, 0, 0, 0, 0},
        {0, 1, 0, 0, 0, 0, 0},
        {0, 0, 1, 0, 0, 0, 0},
        {0, 0, 1, 0, 0, 0, 0}
    };

    BFS(adj);

    return 0;
}
```

Dijkstra

```
#include<iostream>
```

```
#include<omp.h>
```

```
using namespace std;
```

```
#define V 6
```

```
int minDistance(int dist[], bool sptSet[]){
    int min = INT_MAX, min_index = -1;

    #pragma omp parallel
    {
        int local_min = INT_MAX;
        int local_index = -1;

        #pragma omp for nowait
        for(int v=0; v<V; v++){
            if(!sptSet[v] && dist[v] <= local_min){
                local_min = dist[v];
                local_index = v;
            }
        }

        #pragma omp critical
        {
            if(local_min < min){
                min = local_min;
                min_index = local_index;
            }
        }
    }
    return min_index;
}
```

```
void printsolution(int dist[]){
    cout<<"Vertex \t Distance from source\n";
    for(int i=0; i<V; i++){
        cout<<" "<<i<<" \t\t "<<dist[i]<<endl;
    }
}
```

```
void dijkstra(int graph[V][V],int src){
```

```

int dist[V];
bool sptSet[V];

for(int i=0; i<V; i++){
    dist[i] = INT_MAX;
    sptSet[i] = false;
}

dist[src] = 0;

for(int count=0; count<V-1; count++){
    int u = minDistance(dist,sptSet);
    if(u==-1) break;

    sptSet[u] = true;

    #pragma omp parallel for
    for(int v=0; v<V; v++){
        if(!sptSet[v] && graph[u][v] && dist[u] != INT_MAX && dist[u] + graph[u][v]
< dist[v]){
            dist[v] = dist[u] + graph[u][v];
        }
    }
}

printsolution(dist);
}

int main(){
    int graph[V][V] = {
        {0,4,4,0,0,0},
        {4,0,2,0,1,0},
        {4,2,0,1,0,0},
        {0,0,1,0,0,2},
        {0,1,0,0,0,4},
        {0,0,0,2,4,0}
    };

    dijkstra(graph,0);
}

```

Matrix mul

```
#include <iostream>
#include <cstdlib>
#include <omp.h>
using namespace std;

int size1, size2, size3;

void print_mat(int **arr, int size){
    for(int i=0; i<size; i++){
        for(int j=0; j<size; j++){
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl << endl;
}

void matmul(int **arr1, int **arr2){
    #pragma omp parallel for
    if(size1 == size2){

        size3 = size1;
        int **arr3;
        arr3 = new int *[size3];
        for(int i=0; i<size3; i++){
            arr3[i] = new int[size3];
        }

        for(int i=0; i<size1; i++){
            #pragma omp parallel for
            for(int j=0; j<size2; j++){
                #pragma omp parallel for
                for(int k=0; k<size3; k++){
                    #pragma omp parallel for
                    arr3[i][j] += arr1[i][k] * arr2[k][j];
                }
            }
        }

        print_mat(arr3, size3);
    }else{
        cout << "\nSize of the two matrices are not compatible for multiplication.\n";
        return;
    }
}
```

```

    }
}

int main(){
    cout << "Enter the size of 1st square matrix: ";
    cin >> size1;

    int **arr1;
    arr1 = new int *[size1];
    for(int i=0; i<size1; i++){
        arr1[i] = new int[size1];
    }

    for(int i=0; i<size1; i++){
        for(int j=0; j<size1; j++){
            arr1[i][j] = (rand()%10);
        }
    }

    print_mat(arr1,size1);

    cout << "Enter the size of 2nd square matrix: ";
    cin >> size2;

    int **arr2;
    arr2 = new int *[size2];
    for(int i=0; i<size2; i++){
        arr2[i] = new int[size2];
    }

    for(int i=0; i<size2; i++){
        for(int j=0; j<size2; j++){
            arr2[i][j] = rand()%10;
        }
    }

    print_mat(arr2,size2);

    matmul(arr1,arr2);

    return 0;
}

```

